

Practice Exercise 13: Time-Based OTP (TOTP) — Simplified

Context

We are implementing a simplified Two-Factor Authentication (2FA) system (like Google Authenticator). To log in, a user needs a 6-digit code that changes every 30 seconds.

This relies on **HMAC** (Hash-based Message Authentication Code) and the current **Time**. Since both the Client and Server share a `SECRET_KEY` and know the time, they can generate the same code independently.

The Logic (Simplified)

1. **Time Counter (T_c):** Divide the current Unix Timestamp by 30 to get the time window index.

$$T_c = \text{floor}\left(\frac{\text{CurrentUnixTime}}{30}\right)$$

2. **HMAC Generation:** Sign the string representation of T_c using the shared key.

Digest = HMAC-SHA256(key = `SECRET_KEY`, msg = `String( $T_c$ )`)

3. **Code Extraction:** To convert the long Hash Digest into a simple 6-digit PIN:

1. Take the **last 8 hex characters** of the digest (which represents the last 4 bytes).
2. Convert this hex string into a standard Integer.
3. Take the result modulo 1,000,000 to get the 6-digit code.

Your Task

Implement the client-side token generation and the server-side verification.

Required functions

You must implement **two** functions using Python's `hashlib` and `hmac`:

```
def generate_totp(secret_key, current_timestamp):  
    """  
    input:  
        secret_key: str  
        current_timestamp: int  
  
    output:  
        str: The 6-digit string (e.g., "012345")
```

Logic:

1. Calculate `time_counter = current_timestamp // 30`.
 2. Convert `time_counter` to string and encode to bytes.
 3. Compute HMAC-SHA256 using `secret_key` (encoded).
Use `.hexdigest()` to get the hex string.
 4. Take the LAST 8 characters of the hex digest.
 5. Convert that hex slice to an integer (base 16).
 6. Return `str(integer % 1000000).zfill(6)`.
- """

```
def verify_totp(secret_key, user_token, current_timestamp):
```

"""

input:

`secret_key: str`
`user_token: str`
`current_timestamp: int`

output:

`boolean: True if match, False otherwise.`

Logic:

1. Generate the expected token using '`generate_totp`' with the provided timestamp.
 2. Compare expected token vs `user_token`.
 3. Return True if identical.
- """

Dataset

Use `import hmac, import hashlib, import time`.

Test Case 1: Manual Check

Inputs:

- SECRET_KEY: "STUDENT_ID_SECRET"
- Timestamp: 1700000000

Debugging Data:

- `key_bytes = SECRET_KEY.encode('utf-8')`
- **Time Counter:** 56666666
- **HMAC Hex (Last 8 chars):** 87b71d59
- **Int Value:** 2276924761

Expected Output (OTP):

924761

Test Case 2: Live System Verification

Script Logic:

1. Get `current_time = int(time.time())`.
2. Generate a token using a custom key (e.g., "MY_KEY").
3. Immediately call `verify_totp(key, token, current_time)`.
4. Print "PASS" if it returns True.
5. Wait 5 seconds (`time.sleep(5)`).
6. Call `verify_totp` again with the *new* current time.
Note: It should likely still PASS unless the 30-second window just flipped.

Deliverables

A .py file containing:

1. The `generate_totp` function.
2. The `verify_totp` function.
3. Code running Test Case 1 and printing the generated OTP (Must match the expected value).
4. Code running Test Case 2 (Live Check) demonstrating a successful verification.